

**Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии**

ПРАКТИЧЕСКАЯ РАБОТА № 4

**Оценка эффективности статического подавления эквивалентных мутантов
в фреймворке Major: сравнительный анализ режимов с фильтрацией EMS
и без неё**

по дисциплине «Методы тестирования программного обеспечения»

Выполнил
студент гр.5130903/40003

Е.В. Жуковский

Руководитель
старший преподаватель
ВШ ПИ

В.А. Пархоменко

СОДЕРЖАНИЕ

Введение	3
1 Методы.....	4
1.1 Математическая постановка проблемы	4
1.1.1 Основные определения	4
1.1.2 Критерии эквивалентности.....	4
1.1.3 Классификация по сложности обнаружения (ASH).....	5
1.1.4 Постановка задачи подавления	5
1.1.5 Метрики сравнения режимов	5
1.2 Иллюстративный пример	6
1.3 Описание разработанного программного обеспечения (RPN-калькулятор и тесты).....	7
1.4 Описание средств тестирования.....	8
1.4.1 Описание фреймворка Major	8
1.4.2 Описание Equivalent Mutant Suppression	11
1.5 Псевдокод алгоритма EMS	12
1.6 Пошаговое выполнение алгоритма на иллюстративных данных	14
1.7 Установка и настройка ПО, запуск мутационного анализа	15
2 Результаты и обсуждение	17
2.1 Результаты генерации мутантов	17
2.2 Результаты работы EMS	18
2.3 Обсуждение результатов	21
2.3.1 Количество сгенерированных мутантов и качество метрик	21
2.3.2 Временные затраты	22
2.3.3 Сравнение с Trivial Compiler Equivalence (TCE).....	22
2.3.4 Ограничения работы	23
2.3.5 Дальнейшие исследования.....	23
Заключение	24
Список использованных источников.....	25
Приложение 1 Код файла Console.java.....	26
Приложение 2 Код файла FileService.java.....	32
Приложение 3 Код файла RPNCalculator.java	35
Приложение 4 Код файла Main.java	39
Приложение 5 Код файла ConsoleTest.java	40
Приложение 6 Код файла FileServiceTest.java	47

Приложение 7 Код файла RPNCalculatorTest.java.....	52
Приложение 8 Код файла build.xml.....	59

ВВЕДЕНИЕ

Мутационное тестирование является одним из эффективных методов оценки качества разработанных тестов ПО. Суть метода заключается во внесении небольших изменений в тестируемый код (создание мутантов) и в проверке, насколько хорошо реализованные тесты находят эти изменения. Также, уже долгое время существует проблема эквивалентных мутантов - они семантически неотличимы от исходного кода, а значит, их невозможно обнаружить с помощью тестов.

Эквивалентные мутанты портят статистику и метрики адекватности тестирования, тратят ресурсы компьютера и время разработчиков. По информации недавних исследований [2], доля эквивалентных мутантов в проектах на Java составляет около 3%. При этом большая часть таких мутантов возникает из-за типовых паттернов программирования и может быть обнаружена при помощи простого статического анализа.

В данной работе рассматривается и исследуется подход Equivalent Mutant Suppression (EMS), разработанный авторами статьи [2], заключающийся в 10 несложных правилах подавления эквивалентных мутаций во время их генерации. Исходная реализация EMS выполнена для фреймворка мутационного тестирования Java-кода Major.

Цель данной работы - оценить эффективность статического подавления эквивалентных мутантов во фреймворке Major, на примере тестирования приложения «Калькулятор Обратной польской нотации». Сравниваются 2 версии фреймворка Major - со встроенной реализацией EMS и без нее.

Оценка производилась по следующим критериям:

- A. Количество обнаруженных эквивалентных мутантов;
- B. Время генерации мутантов;
- C. Время выполнения мутационного анализа;
- D. Точность подавления.

1 МЕТОДЫ

1.1 Математическая постановка проблемы

1.1.1 Основные определения

Пусть P - исходная программа. Под мутантом M понимается программа, полученная из P путём применения одного или нескольких мутационных операторов $\omega \in \Omega$, где Ω - множество допустимых операторов (замена арифметических операций, изменение условных переходов, удаление операторов и т.д.).

Определение 1 (Убитый мутант). Мутант M считается убитым набором тестов T , если существует тест $t \in T$, такой что:

- А. Выполнение t на P завершается успешно (или с ожидаемым результатом);
- В. Выполнение t на M приводит к сбою (или исключению, неверному результату, зависанию) [2].

Определение 2 (Эквивалентный мутант). Мутант M называется эквивалентным P , если не существует теста t (в принципе), который мог бы отличить M от P [2]. Формально:

$$\forall t \in \mathcal{T} : \text{outcome}(P, t) = \text{outcome}(M, t)$$

Где $\mathcal{T}$ - множество всех возможных тестов (с учётом публичного API программы), $\text{outcome}(P, t)$ - абстрактная функция, которая возвращает наблюдаемый результат выполнения программы P на тесте t с точки зрения внешнего интерфейса (публичного API).

1.1.2 Критерии эквивалентности

Для анализа причин эквивалентности используется RIP-критерий [2]. Для того чтобы мутант был неэквивалентным, необходимо выполнение трёх условий:

- А. R (Reachability) - мутированный код должен быть достижим (выполнение в программе должно доходить до мутировавших строк кода) хотя бы одним тестом;
- В. I (Infection) - выполнение мутированного кода должно изменить состояние программы (инфицировать);

С. Р (Propagation) - инфицированное состояние должно распространиться до наблюдаемого выхода (результата теста).

Эквивалентный мутант нарушает хотя бы одно из этих условий. Обозначим через ε множество эквивалентных мутантов. Для каждого мутанта можно определить «слабое звено»: $type(M) \in \{R, I, P\}$, где $type(M)$ - классификатор причины, указывающий, какое из условий не выполняется.

1.1.3 Классификация по сложности обнаружения (ASH)

Для описания того, насколько сложно обнаружить эквивалентность, используется ASH-критерий [2]:

- А. А (Abstract Syntax Tree) - эквивалентность может быть определена только по структуре AST и доступной атрибутивной информации (типы, константы);
- В. S (State / intraprocedural) - требуется анализ потока данных внутри одной процедуры (def-use цепи, диапазоны значений);
- С. Н (Heap / interprocedural) - необходим анализ кучи, alias-анализ или инварианты классов.

1.1.4 Постановка задачи подавления

Пусть M_{all} - множество всех мутантов, генерируемых фреймворком Major для программы P с использованием всех доступных операторов. Требуется построить фильтр F (набор статических правил), который:

- А. Подавляет (не генерирует) мутантов, принадлежащих множеству ε (эквивалентных) с высокой точностью (минимум ложных срабатываний);
- В. Не подавляет неэквивалентных мутантов;
- С. Работает эффективно по времени.

Эффективность фильтра оценивается через:

- А. Полноту (Recall): $|F(M_{all}) \cap \varepsilon|/|\varepsilon|$;
- В. Точность (Precision): $|F(M_{all}) \cap \varepsilon|/|F(M_{all})|$;
- С. Временные затраты на генерацию и анализ.

1.1.5 Метрики сравнения режимов

Для сравнительного анализа режимов (без EMS / с EMS) были введены следующие метрики:

- A. Общее количество сгенерированных мутантов N_{total} ;
- B. Количество эквивалентных мутантов N_{equiv} (оценивается вручную);
- C. Время генерации мутантов t_{gen} (в миллисекундах);
- D. Время полного мутационного анализа $t_{analyze}$ (в миллисекундах);
- E. Количество живых мутантов после прогона тестов N_{live} ;
- F. Точность подавления - доля подавленных мутантов, которые действительно эквивалентны (оценивается вручную).

Ожидается, что (подпись EMS - метрики со включенным режимом подавления, $noEMS$ - метрики с выключенным режимом подавления):

$$N_{live}^{EMS} < N_{live}^{noEMS}, t_{gen}^{EMS} \approx 1.17 * t_{gen}^{noEMS}$$

(на основе данных [2], где накладные расходы EMS на генерацию составили менее 0.5%).

1.2 Иллюстративный пример

В качестве примера рассмотрим фрагмент кода на Java, реализующий метод, возвращающий абсолютное значение целого числа:

```
public static int abs(int x) {
    if (x < 0) {
        return -x;
5    } else {
        return x;
    }
}
```

Применим мутационный оператор замены оператора сравнения $<$ на $>$ в выражении $x < 0$:

```
// Мутант M1
public static int abs(int x) {
    if (x > 0) {
5        return -x;
    } else {
        return x;
    }
}
```

Этот мутант не эквивалентен, так как мутированный метод для числа 5 вернет -5, а оригинальный метод вернет 5. Тест с любым целым x убьет мутанта.

Применим другой мутационный метод, заменим оператор сравнения $<$ на $\leq$ в выражении $x < 0$:

```
// Мутант M2
public static int abs(int x) {
    if (x <= 0) {
5      return -x;
    } else {
        return x;
    }
}
```

Этот мутант эквивалентен. Очевидно, что для любого целого x оригинальный и мутированный метод вернут один и тот же результат. Следовательно, их поведение считается эквивалентным с точки зрения мутационного анализа. Такой мутант не будет убит ни одним тестом.

Далее рассмотрим пример эквивалентных мутантов, подавляемых правилом equals() Reference Equality (см. Listing 1 в [2] и Example 4 в [1]):

```
public boolean equals(Object other) {
    if (this == other) {
        return true;
5 +    ;
    }
    // ...
}
```

Мутант, заменяющий условие $this == other$ на $false$ (или тело условного оператора), эквивалентен, так как при отсутствии ошибок в оставшейся части метода, поведение мутированной и оригинальной версии идентично. EMS подавляет такие мутанты.

1.3 Описание разработанного программного обеспечения (RPN-калькулятор и тесты)

Для сравнения работы фреймворка Major (с EMS и без EMS) было разработано приложение «Калькулятор Обратной польской нотации» (RPN Calculator) в качестве тестируемой программы. Приложение вычисляет арифметические выражения, записанные в постфиксной форме. Программа поддерживает следующие операции в выражении Обратной польской нотации: сложение, вычитание, умножение, деление, смена знака, а также работа с вещественными числами.

Выбор RPN-калькулятора обоснован следующими фактами:

- A. Наличие чёткой спецификации - легко формализовать ожидаемое поведение приложения;
- B. Разнообразие мутационных операторов - арифметические замены, изменения порядка операндов, удаление операций, изменение условий;
- C. Возможность автоматизированного тестирования - можно сгенерировать большое количество корректных выражений;
- D. Компактность кода - позволяет быстро проводить мутационный анализ и интерпретировать результаты.

Приложение состоит из следующих модулей:

- A. `RPNCalculator` - основной вычислитель, работающий с арифметическим выражением. Содержит 2 метода вычисления: медленный на списке и быстрый на стеке;
- B. `FileService` - модуль для взаимодействия с файловой системой, в частности, с JSON-файлами;
- C. `Console` - модуль, предоставляющий консольный интерфейс пользователю;
- D. `Main` - точка входа в приложение.

Помимо этого, были написаны модульные тесты для каждого модуля (кроме `Main`). Они покрывают основную функциональность калькулятора, включая граничные значения.

Также, был написан файл `build.xml`, обеспечивающий автоматическую компиляцию кода приложения и тестов, проведение мутационного анализа.

1.4 Описание средств тестирования

1.4.1 Описание фреймворка *Major*

`Major` [3] - это фреймворк для мутационного анализа Java-кода, интегрированный в компилятор `OpenJDK javac`. Его основные особенности:

- A. Генерирует мутантов на уровне AST - мутации вносятся в синтаксическое дерево до десахаринга, это позволяет точно отображать мутантов на исходный код;

- В. Все мутанты генерируются и встраиваются в один скомпилированный класс-файл, что исключает накладные расходы на повторную компиляцию.
- С. Поддержка слабого и сильного мутационного анализа;
- Д. Оптимизации: предварительный анализ достижимости и инфицирования, приоритизация тестов по времени выполнения.

Мутационные операторы, которые поддерживает Major [3]:

A. AOR: Замена арифметического оператора

Заменяет бинарный арифметический оператор на совместимые альтернативы.

Примеры:

1. $a + b \rightarrow a - b$
2. $a \% b \rightarrow a * b$

B. COR: Замена условного оператора

Заменяет условный оператор на совместимые альтернативы. Также заменяет атомарные булевы условия на true и false (например, `if(flag)` или `if(isSet())`).

Примеры:

1. $a || b \rightarrow a \&\& b$
2. `if(flag) → if(true)`

C. LOR: Замена логического оператора

Заменяет бинарный логический оператор на совместимые альтернативы.

Примеры:

1. $a \wedge b \rightarrow a | b$

D. ROR: Замена оператора отношения

Заменяет оператор отношения на совместимые альтернативы.

Примеры:

1. $a == b \rightarrow a >= b$

E. SOR: Замена оператора сдвига

Заменяет оператор побитового сдвига на совместимые альтернативы.

Примеры:

1. $a >> b \rightarrow a << b$

F. ORU: Замена унарного оператора

Заменяет унарный оператор на совместимые альтернативы.

Примеры:

1. $-a \rightarrow \sim a$

G. LVR: Замена литерального значения

Заменяет литерал на значение по умолчанию.

Числовой литерал заменяется на положительное число, отрицательное число и ноль. Булев литерал заменяется на его логическое дополнение.

Строковый литерал заменяется на пустую строку.

Примеры:

1. `val = 0` $\rightarrow$ 1 и -1
2. `val < 0` $\rightarrow$ 0 и -val
3. `val > 0` $\rightarrow$ 0 и -val

H. EVR: Замена значения выражения

Заменяет выражение (в остальном неизменённом операторе) на значение по умолчанию.

Примеры:

1. `return a` $\rightarrow$ `return 0`
2. `int a = b` $\rightarrow$ `int a = 0`

I. STD: Удаление оператора

Удаляет (опускает) один оператор следующего типа: `return`, `break`, `continue`, вызов метода, присваивание, префиксный/постфиксный инкремент, префиксный/постфиксный декремент.

Примеры:

1. `return a`
2. `break`
3. `continue`
4. `foo(a,b)`
5. `a = b`
6. `++a`
7. `a--`

Для данной работы используется Major версии 3.0.1 (последняя доступная стабильная версия на момент написания работы) с поддержкой EMS и Major версии 2.2.0 (последняя версия без поддержки EMS). Выбор двух разных версий обусловлен тем, что EMS встроен в алгоритм генерации мутантов и не может быть настроен [3].

Более подробное описание поддерживаемых команд можно найти в документации фреймворка [3].

1.4.2 Описание Equivalent Mutant Suppression

Equivalent Mutant Suppression (EMS) [2] - это набор из 10 статических правил, интегрированных непосредственно во фреймворк Major, начиная с версии 3.0.0. Как было описано в пунктах 1.1.2 и 1.1.3, правила делятся на три категории по критерию RIP и на 3 категории по критерию ASH. Полный перечень правил приведен в таблице ниже.

Таблица 1.1

Мутации, подавляемые правилами EMS

ID	Название	RIP	ASH	Краткое описание
1	Unreachable Code	R	A	Условная компиляция (<code>if(false)</code>), недостижимые <code>default</code> в <code>switch</code> , <code>do-while</code> с <code>break/return</code>
2	Standard Library Contracts	I	A	Сравнения с <code>.length/.size()</code> и <code>-1</code> от <code>indexOf</code>
3	Useless Control Flow	I	A	<code>return</code> в конце <code>void</code> , <code>break</code> в конце <code>switch</code> , <code>continue</code> в конце цикла
4	Mutate Constant Value	I	A	Замена константы на семантически равное значение (<code>0</code> → <code>0</code> , <code>final field</code>)
5	Algebraic Identity	I	A	$x+1 \rightarrow x/1$, $x+0 \rightarrow x-0$, $x \gg y \rightarrow x \gg \gg y$ (при знаковых типах)
6	Mutually Exclusive	I	S	<code>&&</code> → <code>!=</code> при непересекающихся условиях, → <code>==</code> при невозможности обоих ложных
7	Loop-Bounds Check	I	S	$i < X \rightarrow i \neq X$ в цикле <code>for</code> с инкрементом на 1
8	<code>equals()</code> Reference Equality	P	A	Отключение оптимизации <code>this==obj</code> в <code>equals</code>
9	Empty Branches	P	A	Мутация условия, если обе ветви пусты
10	Useless Initialization	P	S	Мутация инициализации локальной переменной, значение которой не читается

1.5 Псевдокод алгоритма EMS

Как говорилось ранее, EMS представлен в виде набора из 10 статических правил, которые применяются на этапе построения абстрактного синтаксического дерева (AST) во время компиляции. Ниже приведён псевдокод общей процедуры подавления и детализация некоторых правил.

Input: AST-вершина N , исходный код P , мутационный оператор ω

Output: boolean: true если мутант был подавлен, иначе false

1.shouldSuppress(N , ω , P):

2.**if** isUnreachable(N) **then**

3. | **return** true

4.**if** isStdLibContractMutation(N , ω) **then**

5. | **return** true

6.**if** isUselessControlFlow(N , ω) **then**

7. | **return** true

8.**if** mutateConstantToSameValue(N , ω) **then**

9. | **return** true

10.**if** isAlgebraicIdentity(N , ω) **then**

11. | **return** true

12.**if** isMutuallyExclusive(N , ω) **then**

13. | **return** true

14.**if** isLoopBoundsCheck(N , ω) **then**

15. | **return** true

16.**if** isEqualsReferenceEquality(N , ω) **then**

17. | **return** true

18.**if** isEmptyBranches(N , ω) **then**

19. | **return** true

20.**if** isUselessInitialization(N , ω) **then**

21. | **return** true

22.**return** false

Детализация правила Mutually Exclusive (isMutuallyExclusive):

```

1.isMutuallyExclusive( $N$ ,  $\omega$ ):

   //  $N$  -- логическое выражение,  $\omega$  заменяет && на != или || на
   ==

2.if  $\omega \notin \{AND\_TO\_NEQ, OR\_TO\_EQ\}$  then
3. |   return false

4.left  $\leftarrow N$ .left;
5.right  $\leftarrow N$ .right;

   // Проверяем, что left и right не могут быть одновременно
   истинны (для &&  $\rightarrow$  !=)
   // или одновременно ложны (для ||  $\rightarrow$  ==)

6.if  $\omega = AND\_TO\_NEQ$  then
7. |   if cannotBothBeTrue(left, right) then
8. | |   return true

9.else if  $\omega = OR\_TO\_EQ$  then
10. |   if cannotBothBeFalse(left, right) then
11. | |   return true
12.return false

```

```

1.cannotBothBeTrue(expr1, expr2):

   // Анализ диапазонов: если expr1 и expr2 представляют
   непересекающиеся условия
   // Пример: expr1 = "x < 0" expr2 = "x > 0"

2.ranges1  $\leftarrow$  getValueRange(expr1);
3.ranges2  $\leftarrow$  getValueRange(expr2);
4.return (intersection(ranges1, ranges2) =  $\emptyset$ )

```

Детализация правила Loop-Bounds Check (isLoopBoundsCheck):

```

1.isLoopBoundsCheck( $N$ ,  $\omega$ ):
    //  $N$  -- условие цикла вида  $i < X$ ,  $\omega$  заменяет  $<$  на  $\neq$ 
2.if  $\neg(\text{isForLoopCondition}(N) \wedge \omega = \text{LT\_TO\_NEQ})$  then
3.    return false

    // Проверяем, что  $i$  инициализируется 0, увеличивается на 1,  $X$ 
    // не меняется
4.if isMonotonicIncrement( $i$ )  $\wedge$  isConstantOrSize( $X$ ) then
5.    return true
6.return false

```

1.6 Пошаговое выполнение алгоритма на иллюстративных данных

Рассмотрим фрагмент быстрого алгоритма вычисления RPN-выражения из класса `RPNCalculator` (метод `fastEvaluate`). В этом методе для обработки операндов используется стек `Deque<Double>`.

```

    public double fastEvaluate(String[] tokens) {
        Deque<Double> stack = new ArrayDeque<>();
        for (String token : tokens) {
5           if (isOperator(token)) {
                // ... извлечение операндов и применение оператора
            } else {
                stack.push(Double.parseDouble(token));
            }
10        }
        // ...
    }

```

Шаг 1. Генерация мутанта с оператором замены границы цикла.

Мутационный оператор ROR (Relational Operator Replacement) может заменить условие в цикле `for`. В данном случае цикл реализован как `for (String token : tokens)`, что эквивалентно итерации по массиву от 0 до `tokens.length-1`. Предположим, что при декомпиляции в AST компилятор преобразует этот цикл в классический `for` с индексом:

```

    for (int i = 0; i < tokens.length; i++) {
        String token = tokens[i];
        // тело цикла
5    }

```

Мутант, предлагаемый оператором ROR: замена `i < tokens.length` на `i != tokens.length`.

Шаг 2. Применение правила EMS «Loop-Bounds Check».

Правило анализирует условие цикла и контекст:

- A. Переменная цикла `i` инициализируется нулём (`int i = 0`);
- B. На каждой итерации `i` увеличивается на единицу (`i++`);
- C. Верхняя граница `tokens.length` не изменяется внутри цикла и является положительным целым числом;
- D. Условие `i != tokens.length` семантически эквивалентно `i < tokens.length` так как `i` никогда не превысит `tokens.length` (при `i == tokens.length` цикл завершится, а при `i > tokens.length` не начнётся из-за монотонного возрастания с шагом 1).

Поскольку все предусловия выполнены, функция `isLoopBoundsCheck` возвращает `true`. Мутант подавляется.

Шаг 3. Генерация мутанта, не подавляемого EMS.

Рассмотрим мутацию внутри метода `applyOperator`:

```
private double applyOperator(double a, double b, String op) {
    switch (op) {
        case "+": return a + b;
        5      case "-": return a - b;
        case "*": return a * b;
        case "/":
            if (b == 0) throw new IllegalArgumentException("Де
                ление на ноль");
            return a / b;
        10     default:
            throw new IllegalArgumentException("Неизвестный оп
                ератор");
    }
}
```

Мутационный оператор AOR заменяет `a + b` на `a - b`. Правила EMS не находят причин для подавления. Мутант генерируется и будет убит любым тестом, в котором используется сложение.

1.7 Установка и настройка ПО, запуск мутационного анализа

Требования к окружению:

- A. Операционная система: Linux или macOS, Windows;
- B. Java Development Kit: OpenJDK 8 и 11 (Major 2.2.0 совместим с Java 8, Major 3.0.1 - с Java 11);
- C. Git.

Шаги по установке ПО:

- A. Скачать архивы с кодом Major 2.2.0 и 3.0.1 по следующей ссылке:
<https://mutation-testing.org/>;
- B. Распаковать архивы в удобную директорию, переименовать их в major-3.0.1 (с новой версией) и major-2.2.0 (со старой версией);
- C. Клонировать репозиторий в удобную директорию:

```
git clone https://github.com/swwwjjw/ems-research.git  
cd ems-research.
```

Шаги по запуску мутационного анализа:

- A. Установить OpenJDK 8 в качестве основной Java;
- B. Очистить репозиторий от файлов сборки:

```
/path/to/major-2.2.0/bin/ant clean;
```
- C. Запустить мутационный анализ без EMS:

```
/path/to/major-2.2.0/bin/ant mutation.test;
```
- D. В рабочей директории появится подробное описание сгенерированных мутантов (папка mutants), файл с логами создания мутантов (mutants.log) и .csv файлы с детализацией покрытия;
- E. Установить OpenJDK 11 в качестве основной Java;
- F. Очистить репозиторий от файлов сборки:

```
/path/to/major-3.0.1/bin/ant clean;
```
- G. Запустить мутационный анализ с EMS:

```
/path/to/major-3.0.1/bin/ant mutation.test;
```
- H. В рабочей директории заменятся файлы с подробным описанием сгенерированных мутантов (папка mutants), файл с логами создания мутантов (mutants.log), файл с логами подавления (suppression.log) и .csv файлы с детализацией покрытия.

2 РЕЗУЛЬТАТЫ И ОБСУЖДЕНИЕ

Эксперименты проводились на компьютере Apple MacBook Air M4. Технические характеристики:

- A. OS: macOS 26.3 Tahoe;
- B. Процессор: Apple M4, 8 ядер, тактовая частота - 4,4-4,5 ГГц;
- C. RAM: 16 ГБ;

2.1 Результаты генерации мутантов

Результаты генерации мутантов представлены в таблице ниже.

Таблица 2.1

Эквивалентные мутанты

ID му- танта	Класс, метод	Исходное выражение	Мутированное выражение
25	Console, manualInput	!algorithm.equals("1")&& !algorithm.equals("2")	!algorithm.equals("1")== !algorithm.equals("2")
42	Console, processJsonFile	!algorithm.equals("1")&& !algorithm.equals("2")	!algorithm.equals("1")== !algorithm.equals("2")
99	RPNCalculator, slowEvaluate	i < list.size()	i != list.size()
106	RPNCalculator, slowEvaluate	operatorIndex == -1	operatorIndex <= -1
141	RPNCalculator, isOperator	token.equals("+") token.equals("-")	token.equals("+") != token.equals("-")
145	RPNCalculator, isOperator	token.equals("+") token.equals("-") token.equals("*")	(token.equals("+") token.equals("-")) != token.equals("*")
149	RPNCalculator, isOperator	token.equals("+") token.equals("-") token.equals("*") token.equals("/")	(token.equals("+") token.equals("-") token.equals("*")) != token.equals("/")

Таблица 2.2

Причины эквивалентности

ID мутанта	Почему эквивалентен?	Был подавлен?
25	Для algorithm значения 1 и 2 дают false в обеих формах, любое другое - true; поведение полностью совпадает.	Нет
42	Аналогично мутанту 25: логика проверки выбора алгоритма остается той же.	Нет
99	В цикле for переменная i растёт по 1 от 0, а list.size() в этом месте не меняется; оба условия эквивалентно ограничивают число итераций.	Да
106	operatorIndex принимает только -1 (не найден оператор) или неотрицательные индексы; значений меньше -1 не бывает.	Нет
141	Токен не может одновременно быть + и -; в таком случае OR и XOR дают одинаковый результат.	Да
145	Условие * взаимоисключающее с +/-, поэтому $A \mid\mid B$ эквивалентно $A \neq B$.	Да
149	Проверка на / взаимоисключающая с +, -, *; итоговая логическая функция не меняется.	Да

2.2 Результаты работы EMS

Сравнительная таблица результатов мутационного анализа представлена ниже. Для исключения "выбросов" замеров времени эксперимент был проведен 25 раз, в таблице указаны средние значения.

Таблица 2.3

Результаты мутационного анализа с EMS и без EMS

Метрика	Без EMS	С EMS
Общее количество сгенерированных мутантов N_{total}	176	172
Количество эквивалентных мутантов N_{equiv}	7	3
Количество живых мутантов после анализа	10	6
Количество ложных подавлений	-	0
Время генерации t_{gen} , мс	617	690
Дисперсия времени генерации t_{gen} , мс ²	81	163
Погрешность (95%) времени генерации t_{gen} , +-мс	4	5
Время мутационного анализа ConsoleTest, мс	1289	1487
Время мутационного анализа FileServiceTest, мс	67	44
Время мутационного анализа RPNCalculatorTest, мс	22	19
Время препроцессинга, мс	2926	3349
Время мутационного анализа $t_{analyze}$, мс	4304	4899
Дисперсия времени мутационного анализа $t_{analyze}$, мс ²	369	716
Погрешность (95%) времени мутационного анализа $t_{analyze}$, +-мс	8	11

Добавление фильтрации EMS в исходный код фреймворка Major привело к следующим результатам:

- А. Общее количество мутантов снизилось на 2.3%;
- В. Оценка покрытия стала выше: 96.51% вместо 94.31%;
- С. Количество эквивалентных мутантов снизилось на 57.14%;
- Д. Ложных подавлений не было зафиксировано - 0%;

Тем не менее, временные затраты увеличились:

- А. Время генерации выросло, в среднем, на 11.83%;
- В. Время мутационного анализа выросло на 13.82%;

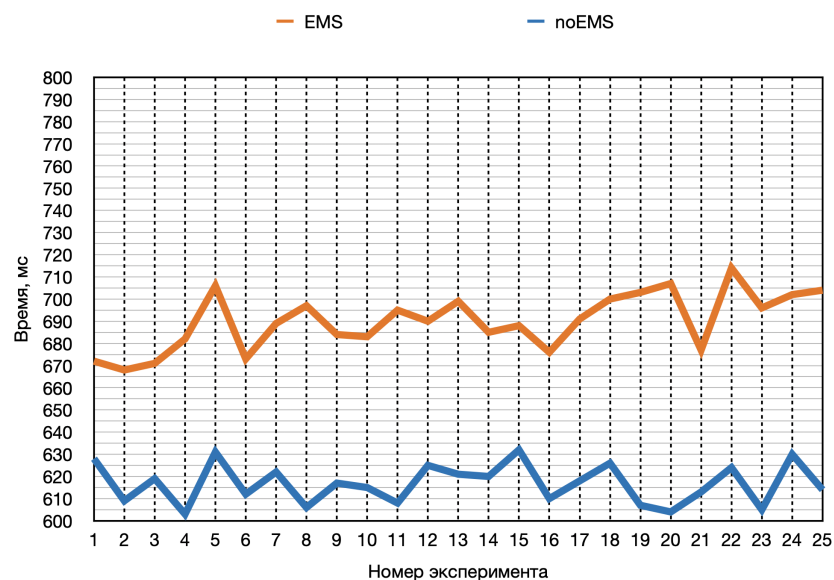


Рис.2.1. Время генерации мутантов

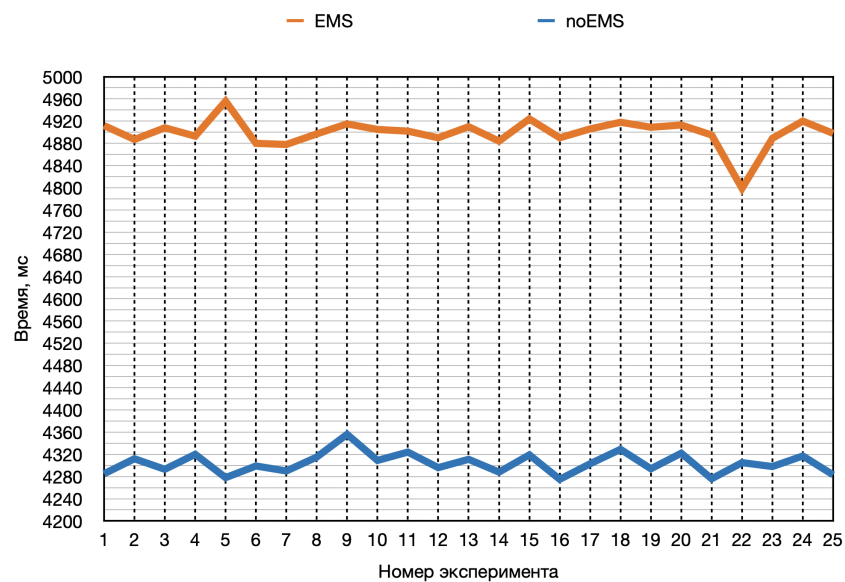


Рис.2.2. Время мутационного анализа

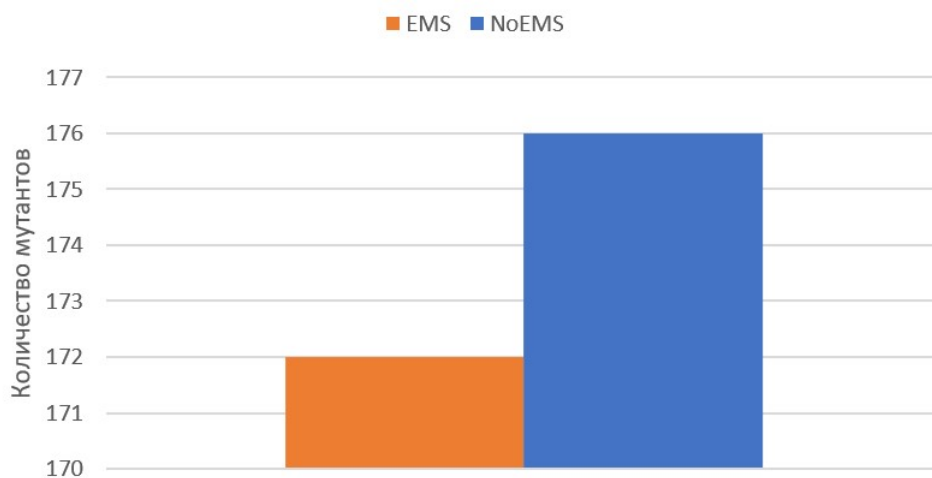


Рис.2.3. Общее количество мутантов

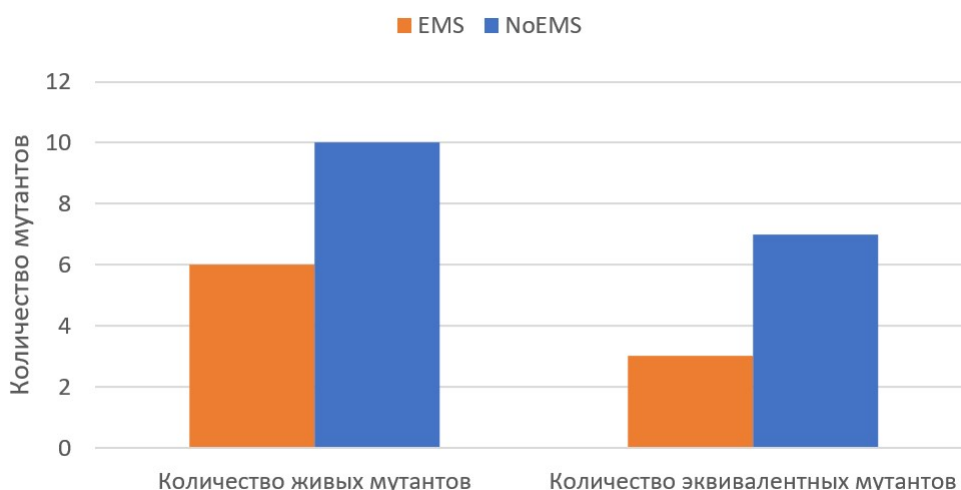


Рис.2.4. Количество выживших и эквивалентных мутантов

2.3 Обсуждение результатов

2.3.1 Количество сгенерированных мутантов и качество метрик

В данном эксперименте включение правил EMS привело к сокращению общего числа сгенерированных мутантов на 4 единицы (со 176 до 172), это снижение примерно на 2.3%. Такой результат хорошо согласуется с оценками, которые были приведены в работе [2], где медианная доля эквивалентных мутантов по 19 Java-проектам составила около 2.97%. Также важно, что из 7 эквивалентных мутантов, выявленных в ходе ручного анализа, EMS успешно подавил 4 (с ID 99, 141, 145 и 149). Оставшиеся 3 эквивалентных мутанта не были распознаны фреймворком, что соответствует ожиданиям: метод нацелен на высокую точность, а не на полноту. Можно заметить, что в данном эксперименте не зафиксировано ни одного ложного подавления - все мутанты, подавленные фреймворком, действительно являлись эквивалентными.

Снижение числа эквивалентных мутантов отразилось на показателе мутационного покрытия (mutation coverage). В режиме без EMS он составил 94.31% (166 убитых из 176), а при использовании EMS - 96.51% (166 убитых из 172). Такой рост вызван удалением «неубиваемых» мутантов, что делает итоговую метрику покрытия более точной. Это наблюдение полностью соответствует тому, что подавление эквивалентных мутантов повышает адекватность мутационного анализа [2].

2.3.2 Временные затраты

Одним из главных преимуществ EMS, заявленных разработчиками, является низкое число временных трудозатрат: в оригинальном исследовании накладные расходы на генерацию мутантов составили менее 0.5% от времени обычной генерации [2]. В данном эксперименте было выявлено заметное увеличение времени генерации - в среднем, с 617 мс до 690 мс (прирост 11.83%). Аналогично, общее время мутационного анализа возросло, в среднем, с 4304 мс до 4899 мс (прирост 13.82%).

Такое расхождение можно объяснить несколькими факторами. Во-первых, абсолютные значения времени в данном эксперименте очень маленькие (менее одной секунды для генерации и менее пяти секунд для полного анализа). В таких условиях даже незначительные постоянные издержки, связанные с инициализацией структур данных для правил EMS или дополнительными обходами AST, существенно влияют на итоговое время. Во-вторых, тестируемое приложение RPN-калькулятор содержит всего 4 класса и около 200 строк кода. На таком малом объёме кода доля времени, потраченная на применение правил подавления, может быть непропорционально велика по сравнению со временем базовых операций компиляции. В крупных проектах, как показано в [2], эти накладные расходы амортизируются и становятся маленькими. В-третьих, следует учитывать разницу в версиях Major: в данном исследовании сравнивались версии 2.2.0 (без EMS) и 3.0.1 (с EMS), которые могут отличаться не только наличием EMS, но и другими параметрами реализации, влияющими на производительность фреймворка.

2.3.3 Сравнение с *Trivial Compiler Equivalence (TCE)*

Прямой экспериментальный запуск TCE в рамках данной работы не производился, но результаты, полученные в [2], позволяют провести косвенное сравнение. В оригинальном исследовании TCE (вариант TCE_{soot}) обнаружил лишь 5.1% эквивалентных мутантов на контрольной выборке, тогда как EMS - около 29%. Также, время обнаружения одного эквивалентного мутанта для TCE_{soot} составило в среднем 4980 секунд, а для EMS - 0,52 секунды. Результаты проведенного мной исследования косвенно подтверждают эти выводы: EMS за счёт интеграции непосредственно в компилятор и использования легковесных статических правил демонстрирует высокую эффективность при минимальных временных затратах.

2.3.4 Ограничения работы

Нужно отметить ряд ограничений, которые имеет данная работа. Во-первых, объём тестируемого кода мал, что, с одной стороны, позволило провести ручной анализ каждого мутанта, но с другой - ограничивает возможность обобщения временных характеристик на промышленные проекты. Во-вторых, сравнение проводилось между двумя разными версиями фреймворка Major (2.2.0 и 3.0.1), что могло внести дополнительные коррективы, не связанные напрямую с EMS. В-третьих, оценка эквивалентности мутантов выполнялась вручную, и это, несмотря на применение критериев RIP, могло привести к ошибкам. Тем не менее, использование строгих определений и дополнительная проверка сомнительных случаев позволили снизить этот риск.

2.3.5 Дальнейшие исследования

Результаты работы открывают несколько перспективных направлений. Во-первых, есть смысл продолжить расширение набора правил EMS для обнаружения тех классов эквивалентных мутантов, которые остались нераспознанными в данном эксперименте. Во-вторых, имеет смысл провести аналогичное исследование на более крупных проектах с открытым исходным кодом для валидации временных затрат. В-третьих, актуальной задачей остаётся интеграция EMS с другими инструментами мутационного тестирования, такими как PIT, где также возникает проблема эквивалентных мутантов [2].

ЗАКЛЮЧЕНИЕ

В рамках данной работы было проведено исследование метода Equivalent Mutant Suppression (EMS), реализованного в фреймворке мутационного тестирования Major, с целью оценки его эффективности при тестировании приложения «Калькулятор Обратной польской нотации». В ходе эксперимента были получены следующие результаты:

- А. Применение EMS позволило сократить общее количество генерируемых мутантов на 2.3%, при этом количество эквивалентных мутантов уменьшилось на 57.14% (с 7 до 3). Ложных подавлений зафиксировано не было;
- В. Мутационное покрытие тестового набора возросло с 94.31% до 96.51%, что говорит о повышении точности метрик адекватности тестирования;
- С. Временные затраты на генерацию мутантов увеличились на 11.83% (с 617 мс до 690 мс), а общее время мутационного анализа - на 13.82% (с 4304 мс до 4899 с). Данное увеличение объясняется маленьким размером тестируемой программы и постоянными накладными расходами на применение правил EMS, которые в более крупных проектах, если основываться на данных из [2], становятся пренебрежимо малыми.

Проведённый анализ подтверждает, что EMS является эффективным и точным инструментом подавления эквивалентных мутантов, который может быть использован в практических задачах мутационного тестирования Java-приложений. Полученные результаты практически полностью совпадают с данными оригинального исследования [2] и демонстрируют применимость подхода на примере компактного, но содержательного программного проекта.

Разработанное программное обеспечение (RPN-калькулятор с модульными тестами) и конфигурация для запуска мутационного анализа могут служить основой для дальнейших исследований в области оценки качества тестов и подавления эквивалентных мутаций.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Equivalent mutant suppression in Java programs. — URL: <https://uwplse.org/2025/01/06/ems.html> (visited on 28.03.2026).
2. Equivalent Mutants in the Wild: Identifying and Efficiently Suppressing Equivalent Mutants for Java Programs / B. Kushigian [et al.] // Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA'24). — Vienna, 2024. — P. 654–665. — DOI [10.1145/3650212.3680310](https://doi.org/10.1145/3650212.3680310).
3. The Major Mutation Framework. — URL: <https://mutation-testing.org> (visited on 27.03.2026).

Код файла Console.java

```
package com.example;

import java.util.Scanner;

5 public class Console {
    private final FileService fileService;
    private final RPNCalculator calculator;

10 /**
     * Конструктор класса Console.
     * @param fileService сервис для работы с JSON-файлами
     * @param calculator калькулятор для вычисления выражений
     * в RPN
     */
15 public Console(FileService fileService, RPNCalculator
        calculator) {
    this.fileService = fileService;
    this.calculator = calculator;
}

20 /**
     * Главный метод запуска консольного интерфейса.
     * Отображает меню и обрабатывает выбор пользователя в бес-
       конечном цикле до выхода.
     * Использует try-with-resources для автоматического закры-
       тия Scanner.
     */
25 public void run() {
    try (Scanner scanner = new Scanner(System.in)) {
        while (true) {
            printMenu();
            String choice = scanner.nextLine().trim();
30 switch (choice) {
                case "1":
                    manualInput(scanner);
                    break;
                case "2":
                    processJsonFile(scanner);
                    break;
35 case "3":
```

```

        showHelp();
        pause(scanner);
        break;
40     case "4":
        System.out.println("Выход из программы
            .");
        return;
        default:
45     System.out.println("Неверный выбор. По
            пробуйте снова.");
    }
}
}
}
50
/**
 * Выводит на экран главное меню программы.
 */
private void printMenu() {
55     System.out.println("\n===== ГЛАВНОЕ МЕНЮ
        =====");
        System.out.println("1. Ручной ввод выражения");
        System.out.println("2. Обработать JSON-файл");
        System.out.println("3. Показать справку");
        System.out.println("4. Выход");
60     System.out.print("Выберите пункт: ");
}

/**
 * Обработывает ручной ввод выражения пользователем.
65 * Запрашивает выражение в RPN, выбор алгоритма, вычисляет
    результат и выводит его.
 * @param scanner объект Scanner для чтения ввода с консол
    и
 */
public void manualInput(Scanner scanner) {
    System.out.println("\n--- Ручной ввод выражения ---");
70     System.out.print("Введите выражение в обратной польско
        й нотации (токены через пробел): ");
        String expression = scanner.nextLine().trim();
        if (expression.isEmpty()) {
            System.out.println("Выражение не может быть пустым
                .");
            return;
75     }
}

```

```

System.out.print("Выберите алгоритм (1 - медленный, 2
- быстрый): ");
String algorithm = scanner.nextLine().trim();
if (!algorithm.equals("1") && !algorithm.equals("2"))
{
80     System.out.println("Неверный выбор алгоритма.");
    return;
}

try {
85     String[] tokens = expression.split("\\s+");
    double result;
    if (algorithm.equals("1")) {
        result = calculator.slowEvaluate(tokens);
    } else {
90         result = calculator.fastEvaluate(tokens);
    }
    System.out.println("Результат: " + formatResult(
        result));
} catch (IllegalArgumentException e) {
    System.out.println("Ошибка вычисления: " + e.
        getMessage());
95 }
}

/**
 * Обработывает JSON-файл: читает выражение из входного фа
    йла,
100 * вычисляет его указанным алгоритмом и сохраняет результа
        т в выходной JSON-файл.
    * @param scanner объект Scanner для чтения путей к файлам
        и выбора алгоритма
    */
public void processJsonFile(Scanner scanner) {
    System.out.println("\n--- Обработка JSON-файла ---");
105 System.out.print("Введите путь к входному JSON-файлу:
        ");
    String inputPath = scanner.nextLine().trim();

    System.out.print("Введите путь к выходному JSON-файлу
        (Enter для выхода в текущую папку): ");
    String outputPath = scanner.nextLine().trim();
110 if (outputPath.isEmpty()) {
        outputPath = "output.json";
    }
}

```

```

    }

    System.out.print("Выберите алгоритм (1 - медленный, 2
    - быстрый): ");
115 String algorithm = scanner.nextLine().trim();
    if (!algorithm.equals("1") && !algorithm.equals("2"))
    {
        System.out.println("Неверный выбор алгоритма.");
        return;
    }
120
    try {
        String expression = fileService.
            readExpressionFromJson(inputPath);
        String[] tokens = expression.split("\\s+");

125 double result;
        if (algorithm.equals("1")) {
            result = calculator.slowEvaluate(tokens);
        } else {
            result = calculator.fastEvaluate(tokens);
130
        }

        fileService.writeResultToJson(outputPath,
            expression, result);
        System.out.println("Результат успешно записан в "
            + outputPath);
    } catch (Exception e) {
135 System.out.println("Ошибка: " + e.getMessage());
    }
}

/**
140 * Выводит на экран подробную справочную информацию о прог
    рамме:
    * описание обратной польской нотации, допустимые оператор
        ы,
    * форматы JSON-файлов, требования к окружению.
    */
private void showHelp() {
145 System.out.println("\n===== СПРАВКА
        =====");
        System.out.println("Калькулятор обратной польской нота
            ции (RPN)");

```

```

System.out.println("Обратная польская нотация (постфик
    сная) - форма записи,");
System.out.println("в которой оператор следует за опер
    андами. Например:");
System.out.println("    инфиксная:  (3 + 4) * 5");
System.out.println("    постфиксная: 3 4 + 5 *");
System.out.println("Допустимые операторы: + - * /")
    ;
System.out.println("Числа могут быть целыми или дробны
    ми (разделитель точка).");
System.out.println("Отрицательные числа вводятся как о
    бычно: -5 2 *");
System.out.println("Алгоритмы вычисления:");
System.out.println("    Медленный - на основе списка (O(
    n^2)), демонстрационный.");
System.out.println("    Быстрый - на основе стека (O(n))
    , используется на практике.");
System.out.println("Режимы работы:");
System.out.println("    1. Ручной ввод - введите строку
    токенов (через пробел) и");
System.out.println("        выберите алгоритм. Результат
    выводится на экран.");
System.out.println("    2. Обработка JSON файла - чтение
    выражения из файла,");
System.out.println("        вычисление и сохранение резул
    тата в JSON.");
System.out.println("Формат входного JSON файла:");
System.out.println("    { \"expression\": \"5 1 2 + 4 *
    + 3 -\" }");
System.out.println("Формат выходного JSON файла:");
System.out.println("    {");
System.out.println("        \"expression\": \"5 1 2 + 4 *
    + 3 -\",");
System.out.println("        \"result\": 14");
System.out.println("    }");
System.out.println("    (целые числа записываются без де
    сятичной точки)");
System.out.println("Требования:");
System.out.println("    - Java 11 или выше");
System.out.println("    - Библиотека Gson (версия
    2.8.9+) - для работы с JSON");
System.out.println("
    =====")
    ;

```

```

}

```

```

175  /**
    * Приостанавливает выполнение до нажатия пользователем
    *   Enter.
    * Используется после вывода справки, чтобы пользователь у
    *   спел прочитать.
    * @param scanner объект Scanner для чтения нажатия Enter
    */
180 private void pause(Scanner scanner) {
    System.out.print("Для возврата в меню нажмите Enter...
        ");
    scanner.nextLine();
    }
185
    /**
    * Форматирует число с плавающей точкой: если значение цел
    *   ое,
    * возвращает строку без десятичной точки, иначе возвращает
    *   обычное строковое представление.
    * @param result результат вычисления
    * @return отформатированная строка
    */
190 private String formatResult(double result) {
    if (result == (long) result) {
        return String.valueOf((long) result);
    } else {
195         return String.valueOf(result);
    }
    }
    }
}

```


Код файла FileService.java

```

package com.example;

import com.google.gson.Gson;
5 import com.google.gson.JsonObject;
import com.google.gson.JsonParseException;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
10 import java.nio.file.Paths;

/**
 * Сервис для работы с JSON-файлами.
 * Обеспечивает чтение выражения из JSON-файла и запись резуль-
тата вычислений в JSON-файл.
15 */
public class FileService {

    /**
     * Объект Gson для преобразования между JSON и Java-объект-
ами.
20 * Используется стандартная конфигурация без дополнительны-
х настроек.
    */
    private final Gson gson = new Gson();

    /**
25 * Читает выражение в обратной польской нотации из JSON-фа-
йла.
     * Ожидается, что файл содержит JSON-объект с полем "
expression".
     *
     * @param filePath путь к входному JSON-файлу
     * @return строка с выражением (токены через пробел)
30 * @throws IOException если возникла ошибка при чтении фай-
ла
     * @throws JsonParseException если содержимое файла не явл-
яется корректным JSON
     * или в нём отсутствует поле "expression"
    */

```

```

public String readExpressionFromJson(String filePath)
    throws IOException, JsonParseException {
35     Path path = Paths.get(filePath);
        String content = new String(Files.readAllBytes(path));
        JsonObject json = gson.fromJson(content, JsonObject.
            class);
        if (!json.has("expression")) {
            throw new JsonParseException("Отсутствует поле '
                expression' в JSON");
40     }
        return json.get("expression").getString();
    }

    /**
45     * Записывает исходное выражение и результат его вычисления
        в JSON-файл.
        * Формат выходного файла:
        * {
        *     "expression": "исходное выражение",
        *     "result": значение
50     * }
        * Если результат является целым числом (например, 14.0),
            он сохраняется как целое (14),
        * иначе сохраняется как число с плавающей точкой.
        *
        * @param outputPath путь к выходному JSON-файлу (будет со
            здан или перезаписан)
55     * @param expression исходное выражение в обратной польско
            й нотации
        * @param result вычисленный результат
        * @throws IOException если возникла ошибка при записи фай
            ла
        */
    public void writeResultToJson(String outputPath, String
        expression, double result) throws IOException {
60     JsonObject outputJson = new JsonObject();
        outputJson.addProperty("expression", expression);
        // Форматируем результат как целое, если это целое чис
            ло
        if (result == (long) result) {
            outputJson.addProperty("result", (long) result);
65     } else {
            outputJson.addProperty("result", result);
        }
    }

```

```
Files.write(Paths.get(outputPath), gson.toJson(
    outputJson).getBytes());
    }
70 }
```

Код файла RPNCalculator.java

```

package com.example;

import java.util.*;

5  /**
   * Калькулятор для вычисления выражений в обратной польской но-
   * тации (RPN).
   * Поддерживает два алгоритма вычисления: медленный (на основе
   * списка,  $O(n^2)$ )
   * и быстрый (на основе стека,  $O(n)$ ).
10  * Допустимые операторы: +, -, *, /.
   */
public class RPNCalculator {

    /**
15  * Медленный алгоритм вычисления выражения в обратной поль-
   * ской нотации.
   * Работает путём многократного прохода по списку токенов:
   * каждый раз находит
   * первый оператор, заменяет два предшествующих операнда р-
   * езультатом операции,
   * удаляет использованные операнды и сам оператор. Повтора-
   * ем до тех пор,
   * пока в списке не останется один элемент - результат.
20  * Сложность:  $O(n^2)$  в худшем случае.
   *
   * @param tokens массив строк, представляющих токены выраж-
   * ения (числа и операторы)
   * @return результат вычисления
   * @throws IllegalArgumentException если выражение некорре-
   * ктно
25  *
   * (недостаточно операндов, неизвестный оператор,
   * деление на ноль и т.д.)
   */
    public double slowEvaluate(String[] tokens) {
        List<String> list = new ArrayList<>(Arrays.asList(
            tokens));

30  while (list.size() > 1) {
        int operatorIndex = -1;

```

```

35     for (int i = 0; i < list.size(); i++) {
        String token = list.get(i);
        if (isOperator(token)) {
            operatorIndex = i;
            break;
        }
    }

40     if (operatorIndex == -1) {
        throw new IllegalArgumentException("Некорректн
            ое выражение");
    }
    if (operatorIndex < 2) {
        throw new IllegalArgumentException("Недостаточ
            но операндов для оператора");
45     }

    double a, b;
    try {
        b = Double.parseDouble(list.get(operatorIndex
            - 1));
50         a = Double.parseDouble(list.get(operatorIndex
            - 2));
    } catch (NumberFormatException e) {
        throw new IllegalArgumentException("Некорректн
            ое выражение");
    }
    String op = list.get(operatorIndex);
55

    double result = applyOperator(a, b, op);

    list.set(operatorIndex - 2, Double.toString(result
        ));
    list.remove(operatorIndex);
60     list.remove(operatorIndex - 1);
    }

    return Double.parseDouble(list.get(0));
}

65 /**
 * Быстрый алгоритм вычисления выражения в обратной польск
 * ой нотации.
 * Использует стек: при чтении числа оно помещается в стек
 * , при чтении оператора

```

```

70      * извлекаются два верхних числа, применяется оператор, ре-
        зультат помещается обратно.
      * Сложность:  $O(n)$ , где  $n$  - количество токенов.
      *
      * @param tokens массив строк, представляющих токены выраж-
        ения (числа и операторы)
      * @return результат вычисления
      * @throws IllegalArgumentException если выражение некорре-
        ктно
75      *      (недостаточно операндов, неизвестный оператор,
        деление на ноль,
      *      неверное количество элементов в стеке после об-
        работки и т.д.)
      */
public double fastEvaluate(String[] tokens) {
    Deque<Double> stack = new ArrayDeque<>();
80
    for (String token : tokens) {
        if (isOperator(token)) {
            if (stack.size() < 2) {
                throw new IllegalArgumentException("Недост
                    аточно операндов для оператора");
85            }
            double b = stack.pop();
            double a = stack.pop();
            double result = applyOperator(a, b, token);
            stack.push(result);
90        } else {
            try {
                stack.push(Double.parseDouble(token));
            } catch (NumberFormatException e) {
                throw new IllegalArgumentException("Некорр
                    ектное выражение");
95            }
        }
    }

    if (stack.size() != 1) {
100        throw new IllegalArgumentException("Некорректное в
            ыражение");
    }
    return stack.pop();
}

105 /**

```

```

110      * Проверяет, является ли токен допустимым оператором.
      *
      * @param token проверяемая строка
      * @return true, если токен равен "+", "-", "*" или "/"; и
      *       иначе false
      */
private boolean isOperator(String token) {
    return token.equals("+") || token.equals("-") || token
        .equals("*") || token.equals("/");
}

115  /**
      * Применяет бинарную операцию к двум операндам.
      *
      * @param a первый операнд
      * @param b второй операнд
120  * @param op оператор в виде строки ("+", "-", "*", "/")
      * @return результат операции a op b
      * @throws IllegalArgumentException если передан неизвестн
      *       ый оператор
      *       или при делении на ноль
      */
125 private double applyOperator(double a, double b, String op
    ) {
    switch (op) {
        case "+": return a + b;
        case "-": return a - b;
        case "*": return a * b;
130  case "/":
            if (b == 0) throw new IllegalArgumentException
                ("Деление на ноль");
            return a / b;
        default:
            throw new IllegalArgumentException("Неизвестны
                й оператор");
135    }
    }
}

```

Код файла Main.java

```
package com.example;

public class Main {
5    /* Точка входа в приложение */
    public static void main(String[] args) {
        FileService fileService = new FileService();
        RPNCalculator calculator = new RPNCalculator();
        Console console = new Console(fileService, calculator)
10        ;
        console.run();
    }
}
```


Код файла ConsoleTest.java

```
package com.example;

import org.junit.After;
5 import org.junit.Before;
import org.junit.Test;
import org.junit.runner.RunWith;
import org.mockito.Mock;
import org.mockito.junit.MockitoJUnitRunner;
10
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.PrintStream;
import java.util.Scanner;
15 import java.io.IOException;

import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.Matchers.containsString;
import static org.junit.Assert.*;
20 import static org.mockito.Mockito.*;

@RunWith(MockitoJUnitRunner.class)
public class ConsoleTest {

25     @Mock
    private FileService mockFileService;

    @Mock
    private Scanner mockScanner;
30

    private final PrintStream originalOut = System.out;
    private ByteArrayOutputStream outputStream;

    /**
35     * Настраивает окружение перед каждым тестом.
     * Перенаправляет стандартный вывод в
     *   ByteArrayOutputStream для последующей проверки.
     */
    @Before
    public void setUp() {
40        outputStream = new ByteArrayOutputStream();
```

```

        System.setOut(new PrintStream(outputStream));
    }

    /**
45     * Восстанавливает стандартный вывод после каждого теста.
    */
    @After
    public void tearDown() {
        System.setOut(originalOut);
50    }

    /**
    * Проверяет ручной ввод корректного выражения с выбором м
        едленного алгоритма.
    * Ожидается вывод результата "7" без сообщений об ошибке.
55    */
    @Test
    public void
        manualInput_validInput_slowAlgorithm_printsResult() {
        RPNCalculator realCalculator = new RPNCalculator();
        Console app = new Console(mockFileService,
            realCalculator);

60        when(mockScanner.nextLine()).thenReturn("3 4 +", "1");

        app.manualInput(mockScanner);

65        String output = outputStream.toString();
        assertThat(output, containsString("Результат: 7"));
        assertFalse(output.contains("Ошибка"));
    }

    /**
70    * Проверяет ручной ввод корректного выражения с выбором б
        ыстрого алгоритма.
    * Ожидается вывод результата "3".
    */
    @Test
75    public void
        manualInput_validInput_fastAlgorithm_printsResult() {
        RPNCalculator realCalculator = new RPNCalculator();
        Console app = new Console(mockFileService,
            realCalculator);

        when(mockScanner.nextLine()).thenReturn("5 2 -", "2");
    }

```

```

80         app.manualInput(mockScanner);

        String output = outputStream.toString();
        assertThat(output, containsString("Результат: 3"));
85     }

    /**
     * Проверяет ручной ввод некорректного выражения (например
     * , "3 +").
     * Ожидается вывод сообщения об ошибке вычисления.
90     */
    @Test
    public void manualInput_invalidExpression_printsError() {
        RPNCalculator realCalculator = new RPNCalculator();
        Console app = new Console(mockFileService,
            realCalculator);

95        when(mockScanner.nextLine()).thenReturn("3 +", "1");

        app.manualInput(mockScanner);

        String output = outputStream.toString();
        assertThat(output, containsString("Ошибка вычисления: "
100            ));
    }

    /**
     * Проверяет, что при вводе пустого выражения выводится со
     * общение о валидации,
     * и дальнейший ввод алгоритма не запрашивается.
105     */
    @Test
    public void
        manualInput_emptyExpression_printsValidationMessage() {
110        Console app = new Console(mockFileService, new
            RPNCalculator());

        when(mockScanner.nextLine()).thenReturn(" ", " ");

        app.manualInput(mockScanner);

115        String output = outputStream.toString();
        assertThat(output, containsString("Выражение не может
            быть пустым. "));
    }

```

```

        verify(mockScanner, times(1)).nextLine();
    }

120
    /**
     * Проверяет, что при выборе несуществующего алгоритма (не
       1 и не 2)
     * выводится сообщение об ошибке, и вычисление не выполняе
       тся.
     */
125
    @Test
    public void
        manualInput_invalidAlgorithm_printsValidationMessage()
    {
        Console app = new Console(mockFileService, new
            RPNCalculator());

        when(mockScanner.nextLine()).thenReturn("3 4 +", "9");
130
        app.manualInput(mockScanner);

        String output = outputStream.toString();
        assertThat(output, containsString("Неверный выбор алго
            ритма. "));
135
    }

    /**
     * Проверяет, что при выборе неверного алгоритма в режиме
       обработки JSON-файла
     * чтение и запись файлов не производятся (методы сервиса
       не вызываются).
140
     */
    @Test
    public void
        processJsonFile_invalidAlgorithm_doesNotReadOrWriteFiles
        () throws Exception {
        Console app = new Console(mockFileService, new
            RPNCalculator());

145
        when(mockScanner.nextLine()).thenReturn("input.json",
            "output.json", "0");

        app.processJsonFile(mockScanner);

        String output = outputStream.toString();

```

```

150         assertThat(output, containsString("Неверный выбор алго
           ритма. "));
           // Дополнительно: методы FileService не вызываются (мо
           жно проверить verifyZeroInteractions,
           // но в данном тесте это не требуется, так как исключе
           ние выкинуто раньше)
       }

155     /**
       * Проверяет, что при ошибке чтения JSON-файла (например,
       * файл не найден)
       * выводится соответствующее сообщение об ошибке.
       */
       @Test
160     public void processJsonFile_readFails_printsError() throws
           Exception {
           Console app = new Console(mockFileService, new
           RPNCalculator());

           when(mockScanner.nextLine()).thenReturn("missing.json"
           , "out.json", "1");
           when(mockFileService.readExpressionFromJson("missing.
           json")).thenThrow(new IOException("file not found")
           );

165           app.processJsonFile(mockScanner);

           String output = outputStream.toString();
           assertThat(output, containsString("Ошибка: file not
           found"));

170     }

       /**
       * Проверяет, что при выборе пункта меню "4. Выход" програ
       * мма корректно завершается.
       */
       @Test
175     public void run_exitOption_terminates() {
           String input = "4\n";
           System.setIn(new ByteArrayInputStream(input.getBytes()
           ));

180           Console app = new Console(mockFileService, new
           RPNCalculator());
           app.run();

```

```

        String output = outputStream.toString();
        assertThat(output, containsString("Выход из программы.
185     "));
    }

    /**
     * Проверяет, что при вводе несуществующего пункта меню (н
     * апример, "9")
     * выводится сообщение о неверном выборе, после чего при с
     * ледующем вводе "4" программа завершается.
190     */
    @Test
    public void
        run_invalidChoiceThenExit_printsRetryMessageAndTerminates
        () {
        String input = "9\n4\n";
        System.setIn(new ByteArrayInputStream(input.getBytes()
195         ));

        Console app = new Console(mockFileService, new
            RPNCalculator());
        app.run();

        String output = outputStream.toString();
200        assertThat(output, containsString("Неверный выбор. Поп
            робуйте снова.));
        assertThat(output, containsString("Выход из программы.
            "));
    }

    /**
205     * Проверяет, что при выборе пункта "3. Показать справку"
        выводится подробная справочная информация,
        * после чего при нажатии Enter возвращаемся в меню, и зат
        ем выход по "4" работает корректно.
        */
    @Test
    public void run_helpThenExit_printsHelpAndReturnsToMenu()
        {
210        String input = "3\n\n4\n";
        System.setIn(new ByteArrayInputStream(input.getBytes()
            ));

```

```
215      Console app = new Console(mockFileService, new  
          RPNCalculator());  
      app.run();  
  
      String output = outputStream.toString();  
      assertTrue(output.contains("===== СПРАВК  
          A ====="));  
      assertTrue(output.contains("Для возврата в меню нажмит  
          e Enter..."));  
      assertTrue(output.contains("Выход из программы.));  
220  }  
    }
```

Код файла FileServiceTest.java

```
package com.example;

import com.google.gson.JsonParseException;
5 import org.junit.After;
import org.junit.Before;
import org.junit.Test;

import java.io.IOException;
10 import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

import static org.hamcrest.MatcherAssert.assertThat;
15 import static org.hamcrest.Matchers.containsString;
import static org.junit.Assert.*;

public class FileServiceTest {

20     private final FileService fileService = new FileService();
    private Path tempDir;

    /**
     * Создаёт временную директорию перед каждым тестом.
     * Все тесты используют изолированное файловое пространство.
25     *
     * @throws IOException если не удалось создать временную директорию
    */
    @Before
30     public void setUp() throws IOException {
        tempDir = Files.createTempDirectory("FileServiceTest");
    }

    /**
35     * Удаляет временную директорию и все её содержимое после каждого теста.
    *

```



```

    * @throws IOException если не удалось удалить файлы или д
    иректорию
    */
    @After
40 public void tearDown() throws IOException {
        // Удаляем временные файлы и директорию
        if (tempDir != null && Files.exists(tempDir)) {
            Files.list(tempDir).forEach(file -> {
                try { Files.delete(file); } catch (IOException
45 ignored) {}
            });
            Files.delete(tempDir);
        }
    }

50 /**
    * Проверяет, что метод readExpressionFromJson корректно и
    * звлекает выражение
    * из правильно сформированного JSON-файла.
    *
    * @throws IOException если возникла ошибка ввода-вывода (
    * не ожидается в тесте)
55 */
    @Test
    public void
        readExpressionFromJson_correctFile_returnsExpression()
        throws IOException {
        Path inputFile = tempDir.resolve("input.json");
        String jsonContent = "{\"expression\": \"3 4 +\"}";
60 Files.write(inputFile, jsonContent.getBytes());

        String expression = fileService.readExpressionFromJson
            (inputFile.toString());
        assertEquals("3 4 +", expression);
    }

65 /**
    * Проверяет, что при отсутствии поля "expression" в JSON-
    * файле
    * выбрасывается JsonParseException с соответствующим сооб
    щением.
    *
    * @throws IOException если возникла ошибка ввода-вывода (
70 не ожидается в тесте)
    */

```

```

@Test
public void
    readExpressionFromJson_missingExpressionField_throwsException
    () throws IOException {
    Path inputFile = tempDir.resolve("missing.json");
75    String jsonContent = "{\"other\": \"value\"}";
    Files.write(inputFile, jsonContent.getBytes());

    try {
        fileService.readExpressionFromJson(inputFile.
            toString());
80        fail("Expected JsonParseException");
    } catch (JsonParseException ex) {
        assertThat(ex.getMessage(), containsString("Отсутс
            твует поле 'expression'"));
    }
}

85
/**
 * Проверяет, что при попытке прочитать некорректный JSON
 * (например, незакрытую фигурную скобку)
 * выбрасывается JsonParseException.
 *
90 * @throws IOException если возникла ошибка ввода-вывода (
 *     не ожидается в тесте)
 */
@Test
public void
    readExpressionFromJson_malformedJson_throwsException()
    throws IOException {
    Path inputFile = tempDir.resolve("bad.json");
95    String jsonContent = "{";
    Files.write(inputFile, jsonContent.getBytes());

    try {
        fileService.readExpressionFromJson(inputFile.
            toString());
100        fail("Expected JsonParseException");
    } catch (JsonParseException e) {
        // Ожидаемое исключение
    }
}

105
/**

```

```

    * Проверяет, что при указании несуществующего файла выбра
      сывается IOException.
    */
@Test
110 public void
    readExpressionFromJson_fileNotFound_throwsIOException()
    {
        String nonExistentFile = tempDir.resolve("no.json").
            toString();
        try {
            fileService.readExpressionFromJson(nonExistentFile
            );
            fail("Expected IOException");
115     } catch (IOException e) {
        // Ожидаемое исключение
    }
}

120 /**
    * Проверяет, что при записи целочисленного результата (на
      пример, 7.0)
    * в выходной JSON-файл значение сохраняется без десятично
      й точки (как целое число).
    *
    * @throws IOException если возникла ошибка ввода-вывода
125 */
@Test
public void writeResultToJson_integerResult_writesInteger
    () throws IOException {
    Path outputFile = tempDir.resolve("output.json");
    String expression = "3 4 +";
130     double result = 7.0;

    fileService.writeResultToJson(outputFile.toString(),
        expression, result);

    String content = new String(Files.readAllBytes(
        outputFile));
135     assertThat(content, containsString("\"result\":7"));
    assertThat(content, containsString("\"expression\": \"3
        4 +\""));
}

/**

```

```

140      * Проверяет, что при записи дробного результата (например
        , 1.5)
        * в выходной JSON-файл значение сохраняется с десятичной
          точкой.
        * Тест пропускается на Windows из-за возможных различий в
          формате десятичного разделителя.
        *
        * @throws IOException если возникла ошибка ввода-вывода
145      */
@Test
public void writeResultToJson_doubleResult_writesDouble()
    throws IOException {
    // Пропускаем на Windows, если платформа Windows - в
    // JUnit 4 используем Assume
    org.junit.Assume.assumeFalse(System.getProperty("os.
        name").toLowerCase().contains("win"));

150    Path outputFile = tempDir.resolve("output.json");
    String expression = "3 2 /";
    double result = 1.5;

155    fileService.writeResultToJson(outputFile.toString(),
        expression, result);

    String content = new String(Files.readAllBytes(
        outputFile));
    assertThat(content, containsString("\"result\":1.5"));
    assertThat(content, containsString("\"expression\":\"3
        2 /\"));
160    }
}

```

Код файла RPNCalculatorTest.java

```

package com.example;

import org.junit.Test;
5 import org.junit.runner.RunWith;
import org.junit.runners.Parameterized;
import org.junit.runners.Parameterized.Parameters;

import java.util.Arrays;
10 import java.util.Collection;

import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.Matchers.containsString;
import static org.junit.Assert.*;
15
/**
 * Параметризованный тест для калькулятора RPN.
 * Проверяет как корректные, так и некорректные выражения.
 * Для каждого набора параметров тестируются оба алгоритма: ме-
   ленный и быстрый.
20 */
@RunWith(Parameterized.class)
public class RPNCalculatorTest {

    private final RPNCalculator calculator = new RPNCalculator
        ();
25

    /**
     * Предоставляет набор параметров для корректных выражений
     * .
     * Каждый параметр: строка с выражением и ожидаемый резуль-
       тат.
     * Покрывание: эквивалентные классы (EP) и граничные значени-
       я (BVA).
30
     *
     * @return коллекция массивов {выражение, ожидаемый резуль-
       тат}
     */
    @Parameters(name = "{index}: {0} -> {1}")
    public static Collection<Object[]> validExpressions() {
35         return Arrays.asList(new Object[][]{

```

```
"3 4 *", 12.0}, // EP-1:  
    простейшее умножение  
{"3 4 * 3 +", 15.0}, // EP-2:  
    несколько операций  
{"3 2.33 *", 6.99}, // EP-3:  
    дробные числа  
{"-10 4 /", -2.5}, // EP-4:  
    отрицательные числа  
{"3 4 * 7 + 3 - 2 /", 8.0}, // EP  
    -5: длинная цепочка  
{"3", 3.0}, // BVA  
    -1: единственное число  
{"3 3 * 2 * 2 / 2 * 2 / 2 * 2 / 2 * 2 / 2 * 2 /  
    / 2 * 2 / 2 * 2 / 2 * 2 / 2 * 2 / 2 * 2 / 2  
    * 2 / 2 * 2 / 2 * 2 / 2 * 2 / 2 * 2 / 2 *  
    2 / 2 * 2 /", 9.0}, // BVA-2: длинное выраж  
ение  
{"3 0 - 0 * 0 +", 0.0}, // BVA  
    -3: операции с нулём  
{"10e300 10e300 *", Double.POSITIVE_INFINITY},  
    // BVA-5: переполнение  
{"2e-17 3e-17 *", 6e-34} // BVA  
    -6: очень малые числа  
  
});  
  
}  
  
// Текущие параметры теста (устанавливаются конструктором)  
private final String expression;  
private final double expected;  
  
/**  
 * Конструктор параметризованного теста для корректных выр  
ажений.  
 * @param expression выражение в обратной польской нотации  
 * @param expected ожидаемый результат  
 */  
public RPNCalculatorTest(String expression, double  
expected) {  
    this.expression = expression;  
    this.expected = expected;  
}  
  
/**  
 * Проверяет, что оба алгоритма (медленный и быстрый) возв  
ращают одинаковый
```

```

65      * корректный результат для заданного выражения в пределах
        погрешности 1e-9.
        * Также проверяется, что результат совпадает с ожидаемым.
        */
@Test
public void testBothAlgorithms_returnSameResult() {
70      // assumeTrue заменён на обычную проверку с сообщением
      assertNotNull(expression);
      assertFalse(expression.trim().isEmpty());

      String[] tokens = expression.split("\\s+");
75      double slowResult = calculator.slowEvaluate(tokens);
      double fastResult = calculator.fastEvaluate(tokens);

      assertEquals(expected, slowResult, 1e-9);
      assertEquals(expected, fastResult, 1e-9);
80      assertEquals(slowResult, fastResult, 1e-9);
}

// ===== Внутренний класс для некорректных
// выражений =====

85 /**
    * Параметризованный тест для некорректных выражений.
    * Проверяет, что оба алгоритма выбрасывают
      IllegalArgumentException
    * с ожидаемой частью сообщения.
    */
90 @RunWith(Parameterized.class)
public static class InvalidExpressionsTest {

    private final RPNCalculator calculator = new
        RPNCalculator();
    private final String expression;
95     private final String expectedMessagePart;

    /**
        * Предоставляет набор параметров для некорректных выр-
          ажений.
        * Каждый параметр: выражение и ожидаемая подстрока со-
          общения об ошибке.
100     *
        * @return коллекция массивов {выражение, ожидаемая по-
          дстрока сообщения}
        */

```

```

@Parameters(name = "{index}: {0} -> {1}")
public static Collection<Object[]> invalidExpressions
() {
    105     return Arrays.asList(new Object[][]{
        {"3 *", "Недостаточно операндов"}, //
        EP-6: недостаточно операндов
        {"3 4 * 5", "Некорректное выражение"}, //
        EP-7: лишний операнд
        {"2 5 %", "Некорректное выражение"},
        // EP-8: неизвестный оператор
        {"a 5 +", "Некорректное выражение"}, //
        EP-9: нечисловой токен
    110     {"7 5 8 10", "Некорректное выражение"}, //
        EP-10: только операнды
        {"- * * * +", "Недостаточно операндов"}, //
        EP-11: только операторы
        {"3 0 /", "Деление на ноль"} //
        BVA-4: деление на ноль
    });
}

115 /**
    * Конструктор параметризованного теста для некорректн
    * ых выражений.
    * @param expression некорректное выражение
    * @param expectedMessagePart ожидаемая часть сообщени
    * я об ошибке
    120 */
    public InvalidExpressionsTest(String expression ,
        String expectedMessagePart) {
        this.expression = expression;
        this.expectedMessagePart = expectedMessagePart;
    }

    125 /**
    * Проверяет, что оба алгоритма выбрасывают
    * IllegalArgumentException
    * с сообщением, содержащим ожидаемую подстроку.
    */
    130 @Test
    public void testBothAlgorithms_throwException() {
        String[] tokens = expression.isEmpty() ? new
            String[0] : expression.split("\\s+");

        // Проверка slowEvaluate

```



```

135         try {
            calculator.slowEvaluate(tokens);
            fail("Expected IllegalArgumentException from
                slowEvaluate");
        } catch (IllegalArgumentException e) {
            assertThat(e.getMessage(), containsString(
                expectedMessagePart));
140     }

    // Проверка fastEvaluate
    try {
        calculator.fastEvaluate(tokens);
145         fail("Expected IllegalArgumentException from
            fastEvaluate");
    } catch (IllegalArgumentException e) {
        assertThat(e.getMessage(), containsString(
            expectedMessagePart));
    }
    }

150 }

// ===== Отдельные специфичные тесты
// =====

/**
155  * Проверяет, что медленный алгоритм корректно обрабатывает
    * выражение,
    * состоящее только из чисел (без операторов) - должно быть
    * исключение.
    */
@Test
public void testSlowEvaluate_noOperator() {
160     String[] tokens = "5 3".split("\\s+");
    try {
        calculator.slowEvaluate(tokens);
        fail("Expected IllegalArgumentException");
    } catch (IllegalArgumentException e) {
165         assertThat(e.getMessage(), containsString("Некорре
            ктное выражение"));
    }
}

/**
170  * Проверяет, что медленный алгоритм выбрасывает исключени
    е,

```

```

    * когда оператору не хватает операндов (например, "3 +").
    */
@Test
public void testSlowEvaluate_insufficientOperands() {
175     String[] tokens = "3 +".split("\\s+");
        try {
            calculator.slowEvaluate(tokens);
            fail("Expected IllegalArgumentException");
        } catch (IllegalArgumentException e) {
180             assertThat(e.getMessage(), containsString("Недоста
                точно операндов"));
        }
    }

    /**
185     * Проверяет, что быстрый алгоритм выбрасывает исключение,
     * когда оператору не хватает операндов (например, "3 +").
     */
    @Test
    public void testFastEvaluate_insufficientOperands() {
190         String[] tokens = "3 +".split("\\s+");
            try {
                calculator.fastEvaluate(tokens);
                fail("Expected IllegalArgumentException");
            } catch (IllegalArgumentException e) {
195                 assertThat(e.getMessage(), containsString("Недоста
                    точно операндов"));
            }
        }

        /**
200     * Проверяет, что быстрый алгоритм выбрасывает исключение,
     * когда после обработки в стеке остаётся более одного зна
        чения
     * (например, "3 4 5 +" -> лишний операнд 3).
     */
    @Test
205     public void testFastEvaluate_extraOperands() {
        String[] tokens = "3 4 5 +".split("\\s+");
        try {
            calculator.fastEvaluate(tokens);
            fail("Expected IllegalArgumentException");
210        } catch (IllegalArgumentException e) {
            assertThat(e.getMessage(), containsString("Некорре
                ктное выражение"));
        }
    }

```

}

}

}

Код файла build.xml

```

<project name="MutationTesting" default="mutation.test"
  basedir=". ">

  <!-- Properties -->
5  <property name="lib.dir"           value="lib"/>
  <property name="src.dir"           value="src/main/java"/>
  <property name="test.dir"          value="src/test/java"/>
  <property name="build.dir"         value="build"/>
  <property name="classes.dir"       value="${build.dir}/
    classes"/>
10 <property name="test.classes.dir" value="${build.dir}/test
    -classes"/>
  <property name="major.jar"         value="/Users/egor/
    Downloads/major/lib/major.jar"/>
  <!-- Path to compiled MML file (use mmlc to generate from
    .mml) -->
  <property name="mml.file"          value="/Users/egor/
    Downloads/major/mml/all.mml.bin"/>
  <!-- Output directories for Major -->
15 <property name="mutants.log"       value="mutants.log"/>
  <property name="summary.file"      value="summary.csv"/>
  <property name="details.file"      value="details.csv"/>

  <!-- Classpaths -->
20 <path id="compile.classpath">
  <fileset dir="${lib.dir}" includes="*.jar"/>
  <fileset dir="/Users/egor/Downloads/major/lib"
    includes="major.jar"/>
</path>

25 <path id="test.classpath">
  <path refid="compile.classpath"/>
  <pathelement location="${classes.dir}"/>
  <pathelement location="${test.classes.dir}"/>
</path>
30 <!-- Clean target -->
  <target name="clean">
    <delete dir="${build.dir}"/>
    <delete file="${mutants.log}"/>
  </target>

```

```

35         <delete file="${summary.file}"/>
           <delete file="${details.file}"/>
        </target>

        <!-- Init target: create build directories -->
40    <target name="init">
           <mkdir dir="${classes.dir}"/>
           <mkdir dir="${test.classes.dir}"/>
        </target>

45    <!-- Compile main code with Major compiler plugin -->
        <target name="compile" depends="init">
           <javac srcdir="${src.dir}"
                destdir="${classes.dir}"
                debug="yes"
50             includeAntRuntime="false"
                classpathref="compile.classpath">

               <!-- Add Major plugin -->
               <compilerarg value="-Xplugin:MajorPlugin mml:${mml
                 .file} export.mutants export.context"/>
55           </javac>
        </target>

        <!-- Compile test code (standard javac, no mutation) -->
        <target name="compile-tests" depends="compile">
60           <javac srcdir="${test.dir}"
                destdir="${test.classes.dir}"
                debug="yes"
                includeAntRuntime="false"
                classpathref="test.classpath"/>
65    </target>

        <!-- Mutation analysis target using Major's JUnit
            extension -->
        <target name="mutation.test" depends="compile-tests">
           <!-- Run mutation analysis with Major's custom junit
            task -->
70           <junit printsummary="false"
                showoutput="false"
                haltonfailure="false"
                mutationAnalysis="true"
                analysisType="preproc_mutation"
75           mutantsLogFile="${mutants.log}"
                summaryFile="${summary.file}"

```

```
mutantDetailsFile="${details.file}"
serializedMapsFile="preprocessing.ser"
timeoutFactor="8"
80  timeoutOffset="0"
    excludeFailingTests="true"
    fork="no"
    debug="true">

85  <!-- Use our compiled classes (with mutants
        embedded) -->
    <classpath refid="test.classpath"/>

    <!-- Run all tests from the test source directory
        -->
    <batchtest fork="false">
90      <fileset dir="${test.dir}">
          <include name="**/*Test*.java"/>
      </fileset>
    </batchtest>
    </junit>
95  </target>

</project>
```